



Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Telecommunications and Artificial Intelligence

Péter Brezovcsik

FACE ATTRIBUTE RECOGNITION USING EFFICIENT NEURAL NETWORKS

SUPERVISOR

Dr. Bálint Gyires-Tóth

BUDAPEST, 2025

Table of Contents

Összefoglaló	4
Abstract	5
1 Introduction.....	6
2 Background	8
2.1 Fundamentals of Deep Learning.....	8
2.1.1 Multi-Layer Perceptron (MLP).....	8
2.1.2 Multi-label Classification	9
2.2 Convolutional Neural Networks	9
2.2.1 Convolutional Layer	9
2.2.2 Receptive Field	10
2.2.3 Pooling Layer.....	11
2.2.4 Group Convolution	11
2.2.5 Depthwise-separable Convolution.....	12
3 Related work	13
3.1 Efficient Architectures.....	13
3.1.1 MobileNet.....	13
3.1.2 ShuffleNet.....	15
3.1.3 EfficientNet.....	16
3.2 Face Attribute Recognition.....	17
4 Proposed Work	19
4.1 Objectives	19
4.2 Models and dataset.....	19
4.3 Hardware and Software	21
4.4 Training Pipeline.....	22
4.5 Evaluation metrics and logging	23
4.6 Experiments	24
4.6.1 Training the classifier heads	25
4.6.2 Complete fine-tuning	30
4.6.3 Correlation between metrics and positive samples.....	32
4.6.4 Inference	35
5 Summary	39
References.....	40

HALLGATÓI NYILATKOZAT

Alulírott **Brezovcsik Péter**, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot/ diplomatervet meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy hitelesített felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2025. 05. 23.

.....
Brezovcsik Péter

Összefoglaló

A mélytanulás területén az elmúlt évtizedben elért jelentős fejlődés lehetővé tette kiváló minőségű számítógépes látásmodellek kifejlesztését, amelyek kiemelkedő teljesítményt nyújtanak olyan feladatokban, mint a képosztályozás, objektumfelismerés, szemantikus szegmentálás és egyéb képfeldolgozással kapcsolatos alkalmazások területén.

Az arcattribútum-felismerés alapvetően egy multi-label osztályozási probléma, amely során egyszerre több, egymást kölcsönösen ki nem záró, emberi arcra jellemző tulajdonságot kell azonosítani képek alapján. Ezek a tulajdonságok széles skálán mozoghatnak, lehetnek akár arckifejezések, fizikai jellemzők, illetve egyéb arcvonások. Ennek számos valós alkalmazása létezik, ugyanakkor sok ilyen alkalmazás erőforrásokban limitált eszközökön működik, vagy valós idejű kiértékelést igényel, ezért elengedhetetlen a hatékony neurális hálózatok alkalmazása.

A korszerűbb neurális hálózatok, például olyan konvolúciós hálózatok mint a ResNet, jelentős eredményeket értek el az arcattribútum-felismerés terén. Ezek a modellek azonban gyakran nagy számú paraméterrel rendelkeznek, ami megnehezíti azok alkalmazását erőforrás-korlátozott környezetekben. További számos figyelemre méltó eredményt értek el azzal is, hogy a modell tervezésébe és betanításába domain-specifikus tudást is felhasználtak.

A szakdolgozatom célja hatékony neurális hálózatok alkalmazása és tesztelése az arcattribútum-felismerés területén, amelyek alacsony memóriaigénnyel és inferencia idővel rendelkeznek, miközben magas pontosságot érnek el, anélkül, hogy azok jelentős mértékben használnának domain-specifikus tudást. Továbbá egy olyan felhőben futtatható pipeline létrehozása, amely biztosítja a modellek konzisztens betanítását egy referencia adathalmazon, valamint a modellek teljesítményének értékelése különböző kiértékelési metrikák alapján, például pontosság és késleltetési idő, az arc pozíciójának vizsgálata a modellek teljesítményére továbbá az így kapott eredmények összehasonlítása egy másik, több paraméterrel rendelkező modellel.

Abstract

Recent advancements in deep learning over the past decade have enabled the development of high-quality computer vision models that are capable of achieving outstanding performance on tasks such as image classification, object detection and semantic segmentation and other downstream tasks.

Face attribute recognition is a multi-attribute classification problem where multiple mutually non-exclusive and even mutually exclusive facial attributes must be predicted simultaneously from face images. These attributes can span across a wide range of categories, from facial expressions, physical attributes and other distinguishing facial elements. This has numerous real-world applications, however, many of these are used on edge devices or require real-time evaluation, therefore, the use of efficient neural networks is critical.

Most of the current and past state-of-the-art neural networks, such as convolutional neural networks like ResNets, achieved significant results in the domain of facial attribute recognition. However, these models often have a large number of parameters, which makes them unsuitable to deploy in resource-limited environments. Many other significant results were achieved by incorporating domain-specific knowledge into the training and designing of these models.

The goal of the thesis is to use and experiment with lightweight neural networks, which have small memory footprint and inference time, while achieving great performance in the domain of facial attribute recognition without incorporating a substantial amount of task-specific knowledge. Developing a cloud-based machine learning training pipeline that ensures consistent model training with a benchmark facial image dataset. Evaluate the performance of the models across various metrics, such as accuracy and inference latency, analyze the effect of face pose transformation and compare the results with a more parameter-heavy model.

1 Introduction

Facial attribute recognition is an image classification task and has gained significant attention because of its widespread applications. It significantly differs from face recognition or face verification, which aims to identify one specific individual. Facial attribute recognition focuses on the extraction of categorical, meaningful facial characteristics by simultaneously predicting several facial attributes from a facial image, such as a person’s physical attributes (hair or eye color, the existence and style of facial hair, etc.), accessories (necklaces, earrings, glasses or hats), demographic attributes (age or gender) or the facial expressions (whether a person is smiling or not, etc.).

Image classification tasks can be divided into several categories based on the nature of the classification task. The simplest form of a classification task is binary classification. In this scenario, the task is to classify the image into one of two categories, where these are mutually exclusive. This includes applications like medical imaging, where the models need to identify benign or malignant skin lesions.

Multi-class classification is based on a similar concept. It labels each image with exactly one category, but from three or more possibilities, where the categories are still mutually exclusive. One of the most notable examples of multi-task classification is object identification, like the ImageNet dataset, which consists of a thousand categories, or person identification.

Multi-label classification is a fundamentally different problem from the other two. In this case, the categories have no mutual exclusivity (it could have but it depends on the specific task), and each image can belong to more than one category. This poses several challenges, like the imbalanced nature of the datasets and the correlation between the labels. In many real-world problems, the labels co-exist in a single image and therefore belong to this category. Example applications also include object detection, for example, identifying objects on the street, like signs, cars, people, etc.

Facial attribute recognition is a multi-label classification problem, which poses several challenges beyond standard multi-label classification. One is related to the different facial poses, lighting conditions and image quality that can arise in a real-world scenario. The distribution of the facial attributes is often heavily imbalanced, samples that contains labels like “bald” or “wearing eyeglasses” are less common then “no beard” or

“smiling”, to reduce the effect of the attribute imbalance sampling techniques and/or loss function adjustments must be utilized. One other challenge is the complex inter-attribute correlation between facial attributes. Some facial attributes represent strong relations, for example, “wearing makeup” and “wearing lipstick”, while others are mutually exclusive, like “male” and “female”. This requires detailed modelling, which requires extended knowledge in the domain. However, newer research that is incorporating state-of-the-art convolutional and/or language models often focuses only on developing model architectures that can implicitly model these inter-attribute relations, but these model architectures are often have large number of parameters and not suitable for resource constrained environments.

The relevance of facial attribute recognition has grown in recent years because many everyday applications have incorporated it. Some medical screening methods incorporate AI-assisted facial analysis [33] that identifies facial attributes that are indicative of health diseases, but these methods can be expanded to health monitoring and even decision-making support. Facial attributes can be used in surveillance systems to enhance face recognition. For example, retailers often use their surveillance systems to capture and analyze their customers’ facial images to provide better, more accurate product recommendations [34], however, these applications pose privacy concerns.

State-of-the-art architectures for facial attribute recognition and convolutional networks can achieve great accuracy but requires many parameters that often can not be used in edge devices, such as mobile phones or IoT devices, which has limited computational resources. Throughout the years many lightweight convolution networks has been developed to counteract of these limitations. MobileNet introduced depthwise-separable convolutions, ShuffleNets took advantage of channel shuffle operations and EfficientNets are uses compound scaling methods to reduce the computational cost.

The goal of this thesis is to experiment with the above-mentioned lightweight convolution architectures in the domain of facial attribute recognition, evaluate their performance across several metrics, like accuracy, model size, inference latency and throughput, investigate the effect of facial alignment and compare the result to a parameter-heavy model, like ResNet-50. In Chapter 2, I will review the basics of Deep-Learning with a focus on convolutional networks. Chapter 3 describes these three lightweight architectures and Chapter 4 presents the implemented machine learning pipeline and the performance analysis of the models.

2 Background

In this section, I review the basic building blocks of deep-learning models, with an emphasis on convolutional neural networks.

2.1 Fundamentals of Deep Learning

2.1.1 Multi-Layer Perceptron (MLP)

One of the most important building blocks of modern neural networks is the multi-layer perceptron. It consists of several fully connected layers that are made up of nodes called neurons, these neurons calculate the weighted sum of its input. The purpose of the MLP is to approximate a function. It receives an input and maps it into an output by using weights or parameters that result in the best function approximation. In a supervised learning scenario, when the network makes a prediction during training, the output and the expected output are compared, this is called the loss function. The goal of the training is to minimize the loss by adjusting the parameters of the network. Gradient descent-based methods are the most popular ones to optimize the network, such methods are Adam or SGD. The gradient of the loss function is calculated with respect to the parameters and then this error is propagated backward layer by layer and the parameters are adjusted to reduce loss. When these error gradients become too big (exploding gradients) or too small (vanishing gradients) the network becomes unstable. To make the network more stable and faster, normalization is used. These methods map the error gradients into a defined range. Such methods are Batch Normalization and Layer normalization. Several other components are used in a neural network, like activation functions. When a neuron calculates the weighted sum of the inputs, its output goes through a non-linear activation function, which decides if the neuron should activate or not [22][36].

The simplest MLPs are the feed-forward neural networks. In these networks, the information flows forward, from input to output, without any loop or recurrent connections. When a feed-forward network contains recurrent connections, it is called a recurrent neural network.

2.1.2 Multi-label Classification

In a multi-label image classification problem, the model should decide, whether a label is present on the image or not. When a model makes its prediction, it outputs an array of probabilities. The role of the threshold is to specify a value at which, if the probability of a given label is bigger, then the label is present in the picture, if it is smaller than it is not present in the image.

There are three types of thresholds. Global threshold, label-wise threshold and instance-wise threshold [9]. A threshold is global when it's a single value and all the labels will be assigned to the image based on this value. A label-wise threshold is when there are different thresholds for each label. A threshold is instance-wise when there are separate thresholds for each sample.

Global, label-wise and instance-wise thresholds can be divided into two further subcategories. Whether a threshold is global, label-wise or instance-wise, it can be fixed or adaptive. A fixed threshold means that the value of the threshold doesn't change in the learning phase of the model. A threshold is adaptive when the value of the threshold changes over time based on the quality of the model's predictions.

2.2 Convolutional Neural Networks

For computer vision tasks, one of the most commonly used neural network architecture is the convolutional neural network. It expects images or three-dimensional inputs and uses Convolution Layers to extract features and then these features are fed into fully-connected layers for classification. In this chapter, I describe the layers and concepts used in a Convolutional Neural Network.

2.2.1 Convolutional Layer

The core element of a CNN is the Convolutional Layer. It extracts features from the input volume, such as shapes and edges and when there are several layers these extracted features become more complex. This layer consists of several learnable filters or kernels, each having a spatial dimension and a depth (mostly the same as the depth of the input). Every kernel will traverse the input volume with a specific stride value and perform an elementwise multiplication between the kernel weights and the corresponding input pixels, then sum it together and output a two-dimensional feature map.

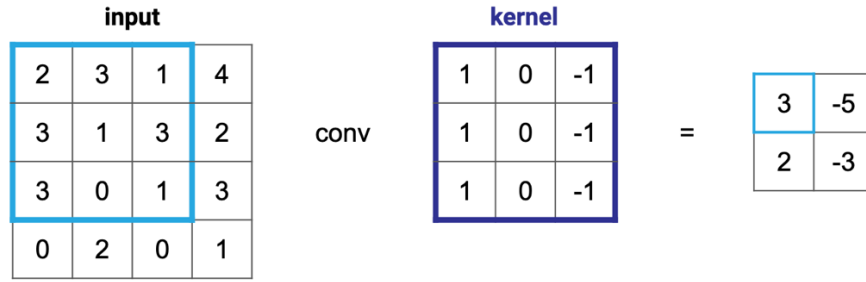


Figure 1: Convolution operation [37]

The number of filters determines the depth or the number of channels/feature maps of the output. The size of the filter controls the spatial dimensions of the output. The stride value indicates how many pixels the kernel shifts at each step. Increasing the stride value is a way of reducing the spatial dimension. The most used filter sizes in practice are 3x3 and 1x1 filters. The latter is also called pointwise convolution, it is used to increase or decrease the number of output channels while preserving the input's spatial dimension. The reason why 3x3 filters are more popular than 5x5 or 7x7 filters is that if two 3x3 filters are stacked the effective receptive field is 5x5 and if three 3x3 filters are stacked the effective receptive field is 7x7, while having fewer parameters [1].

2.2.2 Receptive Field

The receptive field is the region of the input image that the network depends on when creating an output feature [24]. Pixels outside the receptive field do not affect the final feature. Therefore, the receptive field should be controlled to cover all of the relevant area in the input image, this is also called the theoretical receptive field. Because not all pixels in the receptive field contribute equally to the output feature [2], the area of the theoretical receptive field that affects the output feature is called the effective receptive field [2].

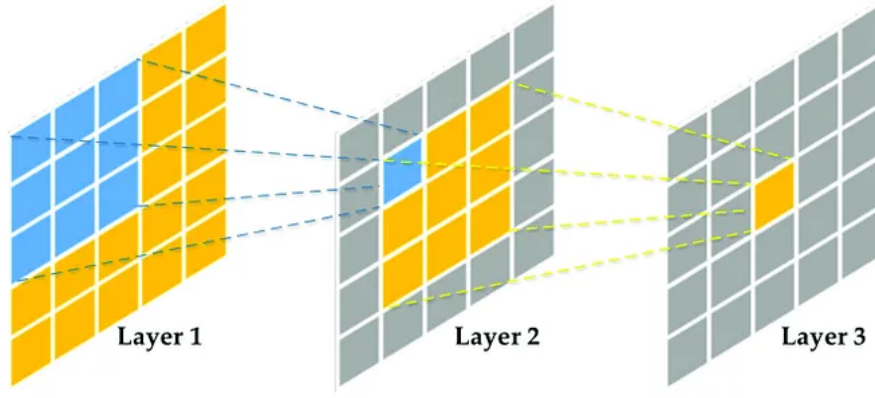


Figure 2: Receptive field [38]

To increase the receptive field, there are several options, one is to increase the number of convolutional layers. Another method is to use pooling layers, dilated convolutions or increase the stride.

2.2.3 Pooling Layer

Pooling is a sub-sampling method whose goal is to reduce the input's spatial dimension but at the same time preserve the relevant information. Like the Convolutional Layer, it also has a filter and a stride, but it does not have any learnable parameters. There are several types of pooling layers, but the most common are the maximum pooling and average pooling. Maximum pooling selects the biggest values from the region that the filter covers. Average pooling averages the values from the area that the filter covers. There is also a pooling technique called global average pooling. This was proposed by Lin. et al. [8], and is used in many convolutional networks to replace or reduce the number of fully-connected layers at the end of the network. This is, in essence, an average pooling layer with a filter size equal to the input's spatial dimension and it produces a scalar value from each feature map. This approach is more robust to spatial translations [8] and overfitting is avoided because there are no learnable weights in this layer.

2.2.4 Group Convolution

To produce a feature map, a filter has to have the same depth as the input volume, this is not an efficient approach when there are several output feature maps. Group convolutions split the input volume into several groups by the input channels and for each group, there is a set of filters that produces a feature map. This approach uses fewer parameters but each output feature map only relates to the input channels within the group

[5]. Group convolutions were originally proposed by Krizhevsky et al. [35] to split the convolutions between two Graphics Processing Unit (GPU). When each input channel forms a group it is called depthwise convolution which is an efficient way of using convolution.

2.2.5 Depthwise-separable Convolution

Depthwise-separable Convolution [11] is a factorization of the classical convolution into two layers, a depthwise convolution and a pointwise convolution. As mentioned in Chapter 2.2.4, a depthwise convolution is when there is a single filter for every input channel of the volume. In this case, the output feature map only relates to one input kernel group. To achieve the same expressive power as a normal convolution, the depthwise convolution is followed by a pointwise convolution which combines the outputs of the depthwise convolution. In this way of splitting the convolution into two parts, we can reduce the model size and amount of computations needed to create an output feature while preserving the same receptive field.

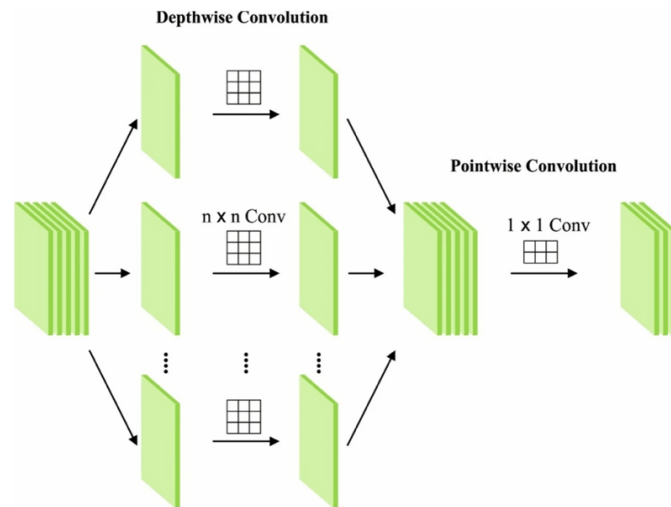


Figure 3: Depthwise-separable convolution [14]

3 Related work

3.1 Efficient Architectures

One disadvantage of convolutional neural networks is that as the number of layers increase, the number of parameters also grow, which results in a large model size that makes it hard to use the high performing, state-of-the art CNNs on resource constrained devices. There are several approaches to obtaining a small network size and lower inference time without the use of specialized computing resources, for example model compression techniques, like quantization [41] or knowledge distillation [19]. In a deep neural network, the weights are stored in 32-bit floating point number format. During quantization the weights are reduced to 16-bit, 8-bit or even smaller floating-point formats, which results in a smaller model size. In knowledge distillation a large, complex model is trained, then it is used to train a smaller network, so the knowledge of the large model is transferred to the smaller one, which is more suitable for deployment. However, my focus is on network architectures, which are efficient in memory consumption and low on inference time. While compression techniques are used during or after model training, efficient networks are designed before training. There are several design choices for creating an efficient architecture. For example, often, the parameters in CNNs have high redundancy and the filters represent similar patterns [15]. This increases the required computation and decreases the learning capacity. For example, to reduce the redundancy of the filters, Jin et al. [15] proposed Flattened CNNs that separate the 3D convolution into three consecutive 1D filters. In this chapter I present efficient network architectures and their structural design choices.

3.1.1 MobileNet

MobileNets are a class of efficient model architectures for mobile and embedded architectures proposed by Howard et al. [11]. It uses 3x3 depthwise separable convolutions to reduce the size of the model and increase speed by using 8 to 9 times less computations than standard convolutions. Every layer, except the last one, is followed by a batchnorm and a ReLU. To reduce the spatial dimension of the image, MobileNet uses strided convolutions in the depthwise layers, and then a final average pooling reduces the spatial dimension into 1x1. From all the layers, the 1x1 convolutions have the most

parameters and account for 95% of the computation time [11]. To further increase the performance of the MobileNets, two global hyperparameters adjust the trade-off between latency and accuracy. One is called the width multiplier, which is a constant that ranges between 0 and 1, with 1 being the baseline MobileNet. Its task is to thin the network by adjusting the number of input and output channels uniformly. It quadratically reduces the number of parameters and computational cost. The other hyperparameter is called the resolution multiplier. It is also a constant between 0 and 1 and reduces the cost of computation quadratically, but in contrast to the width multiplier, its value is set implicitly by setting the input resolution.

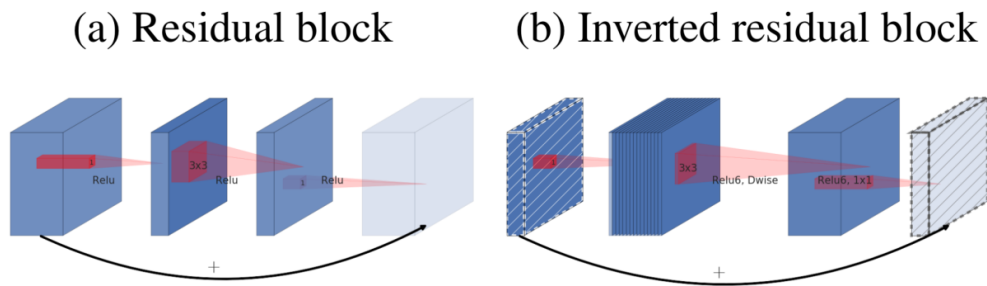


Figure 4: Residual and Inverted Residual blocks [40]

MobileNetV2 proposed by Sandler et al. [10], is an improved version of the MobileNet. It uses a module called an inverted residual block, which is an inversion of the standard residual block. It first increases the number of channels in the input using a 1x1 convolution then applies a 3x3 depthwise convolution and then with another 1x1 convolution, it reduces the channel dimension. At last, the input and output are added together. It reduces memory usage during inference and makes it suitable for mobile applications.

3.1.2 ShuffleNet

ShuffleNet is a highly efficient network architecture proposed by Zhang et al. [5], designed for devices with very limited computational resources. Because pointwise convolutions are expensive, they limit the number of channels in a small network. ShuffleNet uses a pointwise group convolution and the channel shuffle operation to overcome this problem. The usage of the pointwise group convolutions significantly reduces the cost of computation. The channel shuffle operator enables inter-group information flow and also improves accuracy. Its main building block is the ShuffleNet Unit, a residual block that contains channel shuffle and convolution operators.



Figure 5: Channel shuffle and channel split operations. [6]

An improved version of ShuffleNet, proposed by Ma et al. [6], introduces a new operation called channel split, which splits the number of input channels into two parts at the beginning of the ShuffleNet Unit. One branch of the split remains as an identity of the original input, the other goes through convolutions. It also omits pointwise group convolutions for two reasons. One is that channel split already creates two groups, and the other is that extensive group convolutions require fewer FLOPs than dense convolutions but have high memory access cost (MAC) because of the increased number of channels. Because of this high MAC/FLOPs ratio, it also uses fewer element-wise operations like ReLU or AddTensor.

3.1.3 EfficientNet

EfficientNet is an efficient network architecture and a dimension scaling technique [18]. The authors proposed a compound scaling method that uniformly scales the depth, width and resolution of the network. The method uses a user-specified coefficient that controls the remaining resources for scaling and three constants that are determined by a grid search on the original model. These three constants specify how to assign the resources to the different dimensions.

$$\begin{aligned} depth &= \alpha^\phi \\ width &= \beta^\phi \\ resolution &= \gamma^\phi \\ \text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 &\approx 2 \\ \alpha \geq 1, \quad \beta \geq 1, \quad \gamma &\geq 1 \end{aligned}$$

Equation 1: Compound scaling method [18]

This balanced scaling of the network depth, width and resolution leads to better accuracy and efficiency. The authors used neural architecture search to develop a new model architecture to maximize the effectiveness of the model scaling method. The network is composed of inverted residual blocks, which are optimized with squeeze-and-excitation [3]. The base model, EfficientNet-B0, reached 77.1% Top-1 accuracy on the ImageNet dataset with only 5.4 million parameters. For comparison, ResNet-50 with 26 million parameters reached 76.0% Top-1 accuracy on the same dataset.

An upgraded version of the EfficientNet was proposed by Tan and Le [17], named EfficientNetV2. It reduces the training speed and improves parameter efficiency. They used a scaling and neural network search to develop this model family and they also introduced the fused inverted residual block that replaces the pointwise and depthwise convolution to a single 3x3 for scaling up the channel dimension.

3.2 Face Attribute Recognition

Throughout the years, many different methods and architectures have been proposed for facial attribute recognition (FAR) tasks. In this section, I review a few proposed methods.

Most FAR models are trained in a supervised learning fashion. Therefore, they require labeled datasets. These datasets can be laboratory-controlled, where the images are taken in a controlled environment or can be „in the wild” datasets, which are datasets that contain images taken in natural environments. There are many popular datasets for facial attribute recognition, for example, LFW [13] (Labeled Faces in the Wild) contains more than 13,000 images of faces, the UTKFace dataset [26] is a large-scale dataset with over 20,000 face images with annotations of age, gender and ethnicity. FairFace dataset [16] contains more than 100,000 images from seven different race groups, specifically designed for racially fair race group classification. CelebA [21] dataset is another large-scale dataset that contains hundreds of thousands of images labeled with 40 attributes.

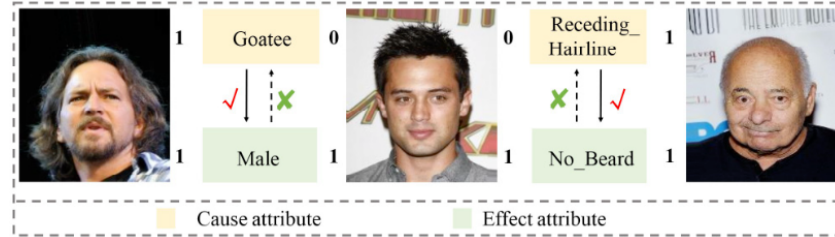


Figure 6: Causal relationship of attributes [7]

Automatic face detection is a prerequisite step for facial attribute recognition, face identification and other similar tasks. In this step, a detection model is used to identify the faces in the image and mark the coordinates of their bounding boxes and optionally, facial landmarks, which are used for face alignment. There are many different face detector models, one popular choice is MTCNN [30]. It is a three-stage cascaded CNN-based lightweight network for face detection. It can also be used for face alignment because the network also predicts five facial landmark locations. RetinaFace [28] is a state-of-the-art single-stage face detector, which simultaneously predicts the face box, a confidence score, five landmarks and 3D position.

Many FAR models are trained in a supervised learning fashion. For example, Sharma and Foroosh [4] proposed a CNN micro-architecture for FAR called Slim-CNN. They incorporated several structural elements into the model, like skip-connection and

multiple branches, while achieving comparable results to those of state-of-the-art models, while using many times fewer parameters and therefore reducing memory requirements and therefore it is suitable for resource limited environments..

FAR models require a large dataset, like the CelebA dataset, but for many applications, the available amount of data is limited. To overcome this problem, [12] used a self-supervised method to extract spatial and semantic information from unlabeled data and then finetuned the model on a limited dataset. To improve the performance of a FAR model and the multi-label classification task in general, many methods exploit the correlation between the labels. Numerous methods use group learning, which is a technique to group attributes with strong correlations. For example, Emily et al. [25] created nine groups of labels based on the location of the attributes. Mao et al. [20] split the facial attributes into two sets, objective and subjective. They also use the facial landmark locations to improve prediction. Instead of correlation, [7] uses the causality of the attributes and classifies them into causal and effect attributes. They also use a wavelet-scattering transformation (WST), which can be seen as a CNN with predetermined, nonlearnable kernels and design a WST block that uses the Squeeze-and-Excitation unit. WST and wavelet-based methods efficiently extract image features in the frequency domain. The MARLIN [31] framework uses a masked autoencoder that utilizes non-annotated facial videos that are available on the internet to learn generic, task-agnostic facial embeddings in a self-supervised way. It can be used as a feature extractor for downstream facial analysis tasks, such as facial attribute recognition and can be deployed on low resource devices like mobile phones.

The modelling of complex attribute relations is a challenging task and often requires domain-specific knowledge. Recently, with the success of NLP models, like the transformer, some frameworks utilize language models for multi-attribute learning. For example, the Label2Label [29] framework achieves competitive results on face attribute recognition. It uses a ResNet-50 as an image feature extractor and an image-conditioned masked language model to learn complex sample-level facial attribute relations.

4 Proposed Work

In this chapter, I provide the results of the experiments conducted on different network architectures with different classifiers.

4.1 Objectives

The goal of the experiments is to see how lightweight CNN architectures can be fine-tuned and used to perform facial attribute recognition and also how the facial alignment affects the results. The models were trained once with a single head classifier and once with a multi-task classifier head. The effect of face alignment was also examined.

4.2 Models and dataset

I used the three previously mentioned models in Section 2, namely ShuffleNetV2, MobileNetV2 and EfficientNet. ResNet-50 was used to compare the results of the models to one that has magnitudes of more parameters. All three models were pretrained on the ImageNet-1k dataset, which contains millions of training images ranging over 1000 classes. Then, a classifier head is attached to the backbone models and these were fine-tuned on the dataset. Table 1 shows the number of parameters of each model.

Table 1: Number of parameters in each model

<i>Model name</i>	<i>Backbone</i>	<i>Classifier head</i>	<i>Model</i>
<i>ShuffleNetV2</i>	341.792	546.344	888.136
<i>MobileNetV2</i>	2.223.872	677.416	2.901.288
<i>EfficientNetB0</i>	4.007.548	677.416	4.684.964
<i>ResNet-50</i>	23.508.032	1.070.632	24.578.664

For the dataset, I chose the CelebA dataset. CelebA dataset is a large-scale dataset with more than 200,000 images of faces and 40 attributes, such as hair color, beard, moustache, etc. The distribution of labels of the CelebA dataset is shown in Figure 7.

One of the major problems of this dataset is that it’s highly imbalanced. A dataset is imbalanced when the distribution of the samples and their corresponding labels are uneven over the data space [27]. There are three types of data imbalance in a multi-label classification problem: imbalance within labels, imbalance between labels, and imbalance among label sets [27]. The CelebA dataset is imbalanced between labels, which means that the number of positive samples for a given label is much higher than the number of positive samples for another label. It is also imbalanced within its labels because many labels contain an extremely high number of negative examples and a low number of positive samples and vice versa.

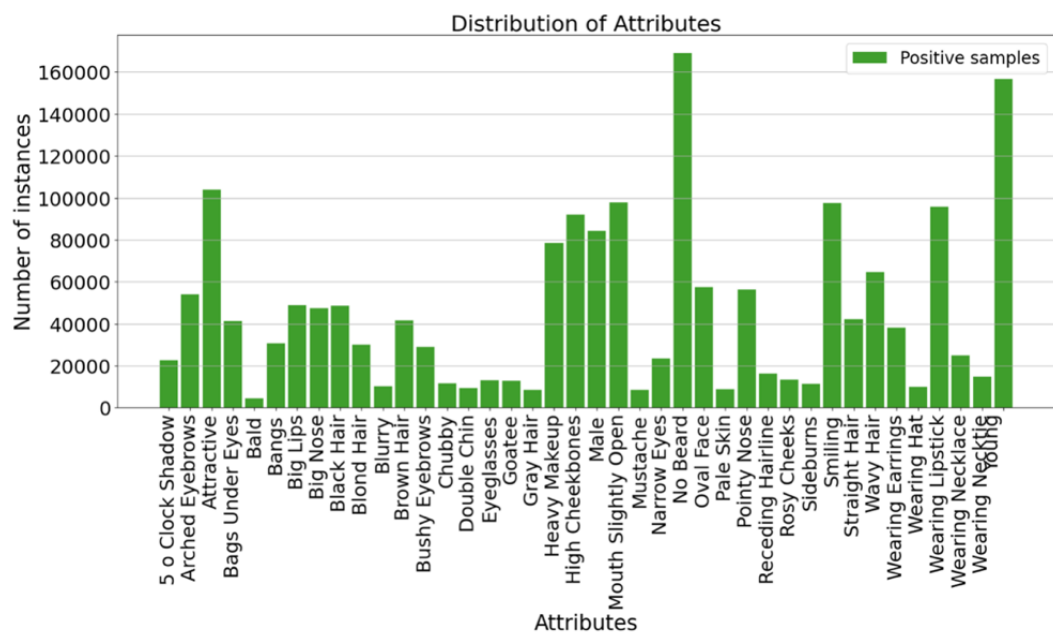


Figure 7: Distribution of positive labels

Commonly used techniques to handle imbalanced datasets are resampling methods. Resampling methods are used during data preprocessing. The two main categories of resampling methods are undersampling and oversampling. Undersampling removes samples from the dataset that are associated with the majority of labels. Oversampling generates samples associated with the minority labels. However, in a

multi-label classification problem, it is not trivial how to use them because, for example, it is not guaranteed that using undersampling, the distribution of the labels would be uniform.

To handle the label imbalance, weights are used in the loss function. The weights are calculated for each attribute, and it is the ratio between the negative examples and the positive examples. This results in a trade-off between recall and precision score, however, the harmonic mean of these values, also known as the F1 score is also higher, which means an overall greater performance.

4.3 Hardware and Software

The models were trained both locally and on an Azure cluster and a similar pipeline was created for each.

Table 2: Computing resources.

	Local	Cloud
Name	MacBook Air	Azure instance
CPU	Apple M1	AMD EPYC 7V12
CPU cores	8	4
RAM	8 GB	28 GB
GPU	Apple M1 GPU	Nvidia Tesla T4
GPU RAM	8 GB	16 GB

I used the Azure Machine Learning service, which is specifically designed for developing and deploying machine learning applications. This service enables the use of IPython notebooks or the creation of ML pipelines that can run in a cloud environment. As the hardware, I created an Azure compute cluster that contained one compute instance, the specifications can be seen in Table 2. Then, the Azure ML pipeline was created, which is similar to that in Figure 8, however, the face detection and facial transformations were in the same component. The pipeline components are Docker containers, and for each component, a Conda environment was created. In the training and testing components, the metrics were logged by MLFlow, which is an open-source platform to manage the lifecycle of a machine learning projects. It includes model tracking, deployment and model versioning and registry. The output of the pipeline was also an MLFlow model, which can be downloaded to a local machine or deployed in Azure ML.

4.4 Training Pipeline

The models were trained both locally and on an Azure cluster and a similar pipeline was created for each.

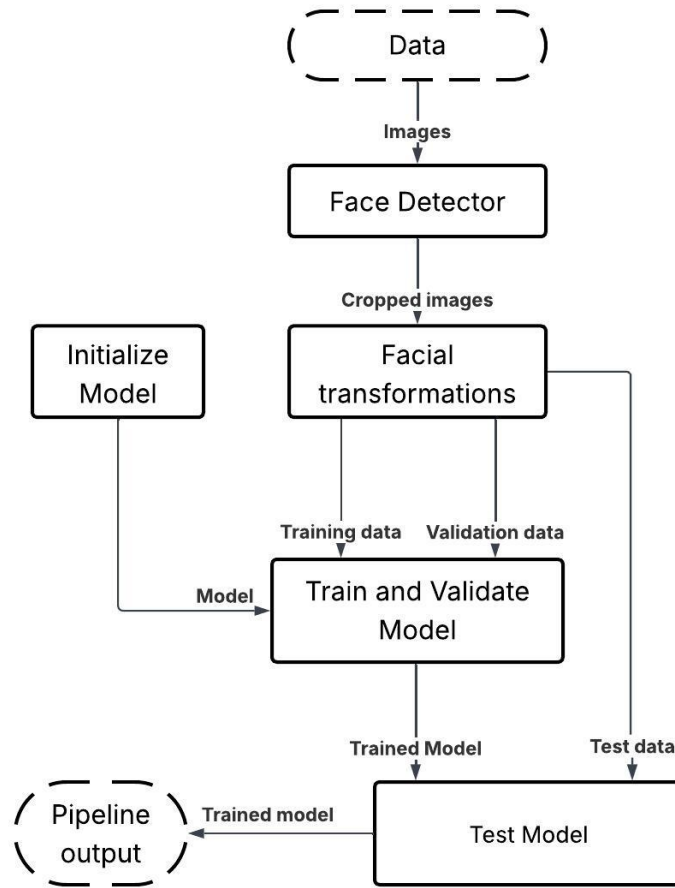


Figure 8: ML training pipeline

As seen in Figure 8, the pipeline contains the following components:

- **Face Detector:** The first component is the face detection step, where the faces are located within an image. The detector provides the bounding box coordinates of these faces and additional facial landmark points. As a face detector, I used a YOLO [32] based model, which was trained to detect faces.
- **Facial Transformations:** This component contains facial or non-model specific transformations, for example face alignment. By separating the face detection and optionally other facial transformations from model-specific transformations results in

faster training because face detection is a much more time-consuming operation than the model-specific ones, and in this way it's enough to do it once, not in each iteration.

- **Initialize Model:** In this component, a model is initialized and passed to the Train Model component. Both random and pretrained model initialization are implemented.
- **Train and Validate Model:** This component contains the model training and validation steps. It also contains the model-specific transformations, which are resizing, normalization and optional data augmentations, like rotation and image flipping.
- **Test Model:** Model testing is the last component, which uses the previously trained model and evaluates it.

4.5 Evaluation metrics and logging

In a classification task, when evaluating the model's output each label can be evaluated as True Positive, True Negative, False Positive or False Negative.

A prediction is a True Positive when the model correctly predicts that the given label is present on the image. True Negative prediction means that the model predicts correctly that a label is not in the image. A prediction is a False Positive when the model incorrectly assigns a label to the image and a False Negative when it incorrectly predicts that the given label is absent from the image. From these four values, further evaluation metrics can be calculated like recall and precision scores. In a multi-label classification problem, all of these scores are calculated for each label and then averaged. I used the following evaluation metrics:

- **Subset Accuracy:** In a multi-label classification task, the subset accuracy is the ratio of the correctly predicted labels to all of the labels. However, by itself, it is not as expressive because, for example, in the CelebA dataset, most of the samples only have a handful of positive labels, which means a high accuracy score can be achieved simply by assigning negative labels to most of the attributes.
- **Label-wise accuracy:** This is an extended version of the subset accuracy, and it is calculated for each label independently.
- **Precision Score:** The precision score is the ratio of the True Positives and the sum of True Positives and False Positives and expresses how many of the predicted positive

labels were actually positive. A higher precision score means that the model is more conservative when predicting positive labels which means that the number of false positive predictions are low.

- **Recall Score:** The recall score is the ratio of the True Positives and the sum of True Positives and False Negatives. A higher recall score means less false negative predictions and it expresses the model's ability to find many positive labels as possible.
- **Hamming Loss:** It is a metric used to evaluate multi-label classification problems. It measures the fraction of incorrect labels to the total number of labels. A lower hamming score means that the classifier predicts the labels correctly.
- **F1-Score:** This is the harmonic mean of the precision score and recall score. It is useful for evaluating imbalanced data. The goal is to achieve a high F1 score, because it means that neither the precision or the recall score is dominate the classifier and therefore its predictions are more balanced.

Both label-wise and averaged precision, recall and F1-scores were calculated. However, when using an imbalanced dataset, the weighted average captures the performance of the model best when the scores are weighted by the number of true instances for each label. This way of calculating these scores accounts for the label imbalance in the dataset.

4.6 Experiments

The models were trained for 25 epochs, without the use of early stopping to ensure fair comparisons to see the convergence of the models and their proneness to overfitting when trained for the same amount of time. The dataset was split into three parts. The training set contained 80% of the dataset, the validation and test datasets each contained 10%. During the face detection step a constant padding was used, which was 20 pixels.

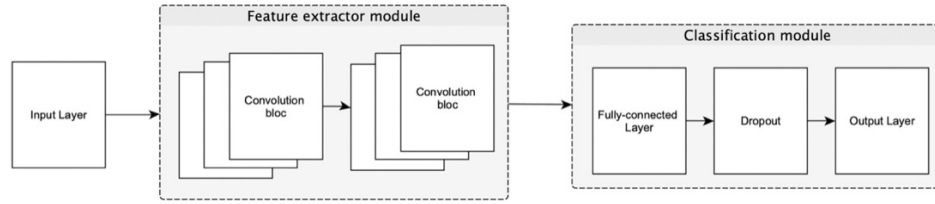


Figure 9: A basic Deep CNN network [23].

The reason for the padding is that the face detector only crops the face and lefts out parts of the head that are necessary to detect facial attributes such as hair, ears.

4.6.1 Training the classifier heads

The first experiments are to see the performance of the models when only the classifier heads are trained. The backbone CNN models are pretrained, but the weights are frozen during training. With this approach, the training is faster in resource-limited platforms.

4.6.1.1 CNN with a single classifier head

At first, I used the three models as feature extractors and then a single classification head was attached to the base model. This means that a model predicts a vector of size 40. The classification model contains a dropout layer, with a probability of 0.2, and two fully-connected layers and a batch-normalization layer. The evaluation metrics on the training, validation and test sets can be seen below.

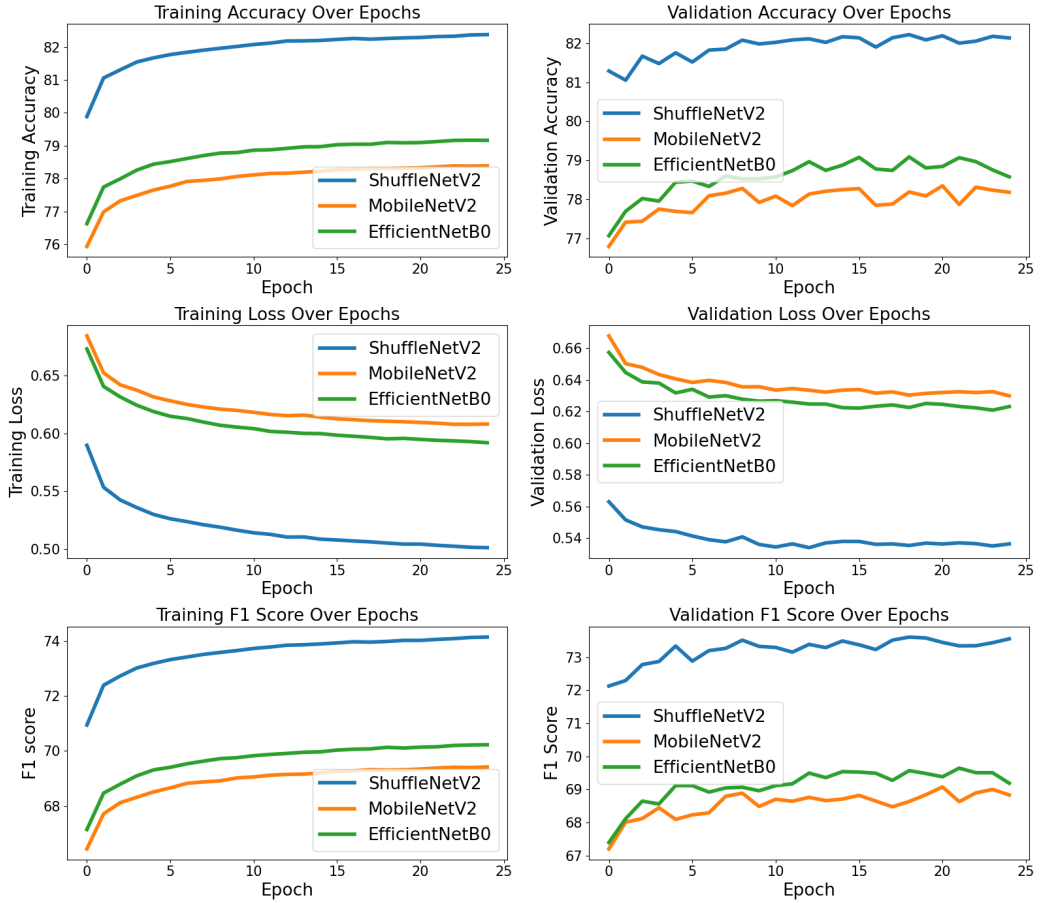


Figure 10: Training and validation curves

For every model, the training curves are smooth and appear to have converged around epoch 15-20. The validation curves show more fluctuation, however, they also converge around 15-20 epochs, and the difference between the training and validation curves indicates that it does not overfit. However, the validation and training curves indicate that further training will not yield any improvement. The accuracy score for every model peaks around 80%. ShuffleNetV2 outperforms both models in every metric, despite having the fewest parameters. The reason for this is that the backbone models are not pre-trained on a facial image dataset but on the same generic one, so when an image is processed, each model looks for relevant features at different locations. However, this is not related to the dataset but to the model architecture. Figure 11 shows the activations of the various models. Only the ShuffleNet has activations in the region of interest, even when the face is previously detected and the image is cropped.

The results, shown in Figure 12, indicate that fine-tuning only the classifier head on the CelebA dataset using these models as feature extractors can lead to limited results and that fine-tuning the backbone models is necessary to achieve acceptable results.

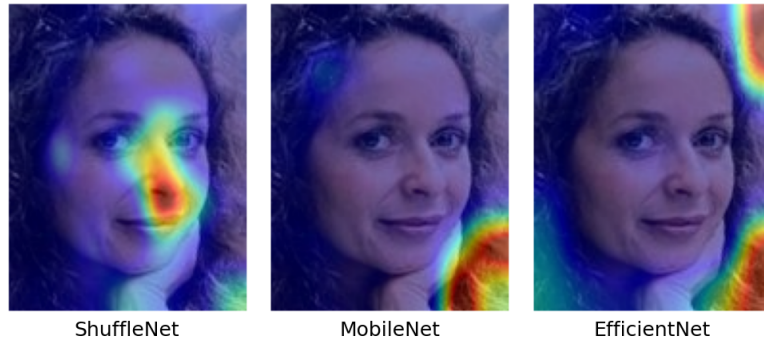


Figure 11: Activation map of different models

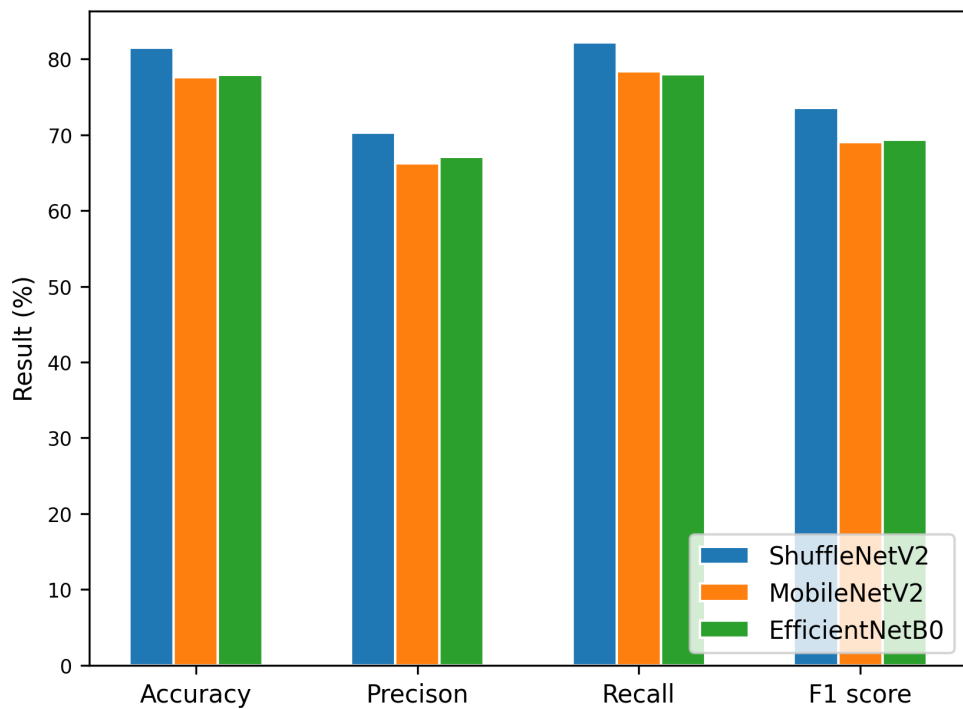


Figure 12: Evaluation metrics on the test set.

4.6.1.2 Using Face Alignment

The second experiment checks the effect of face alignment on facial attribute recognition. To align faces, predefined facial key points are located on the image. These facial landmarks can be varied by the type of face detector. The most typical landmarks are the corners of the eyes, the corners of the mouth and the tip of the nose, but some detectors detect more points or even create a face mesh. After localizing these key points, transformations are applied to the images so that the face is vertical. Figure 13 and Figure 14 shows the label-wise accuracies of the models. Almost every attribute achieved a

slightly greater result when the faces were aligned. Generally, the reason that face attribute recognition, face identification models perform better when the face is aligned is because the backbone CNN models do not have to handle pose variations, because the key facial landmarks are always at the same location. This consistency of facial poses makes it possible for even pretrained models to achieve slightly better results with face alignment than without it. Table 3 shows the evaluation metrics of the models.

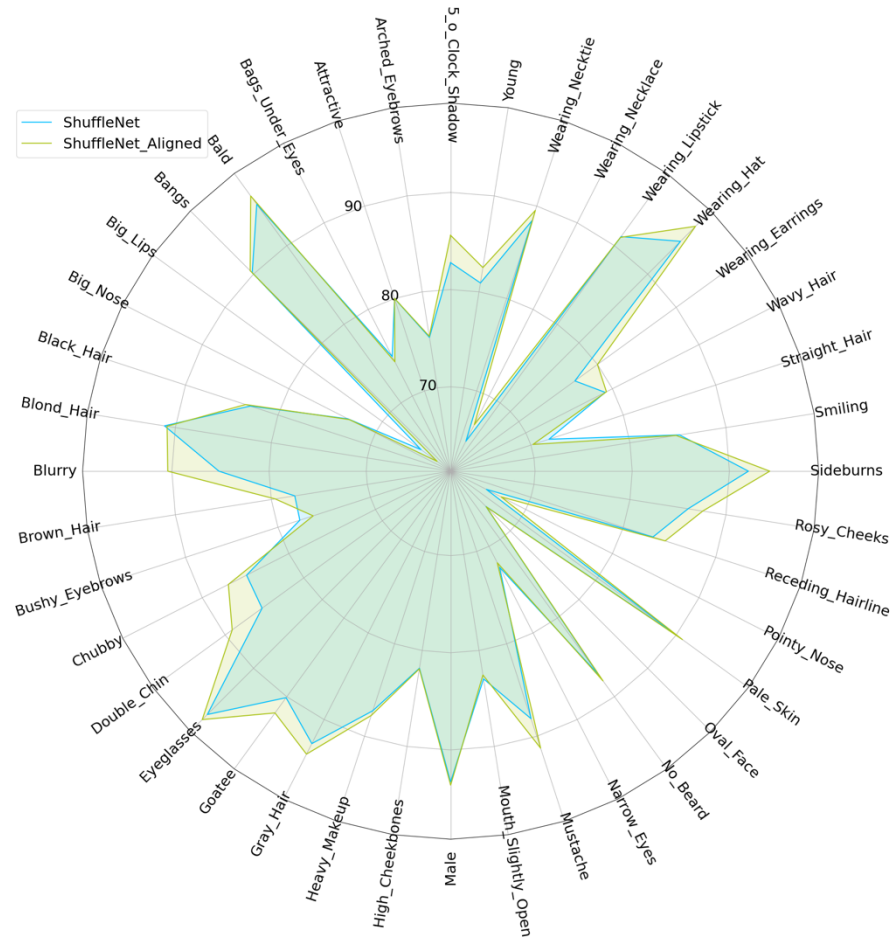


Figure 13: Label-wise accuracies of ShuffleNet with and without face alignment.

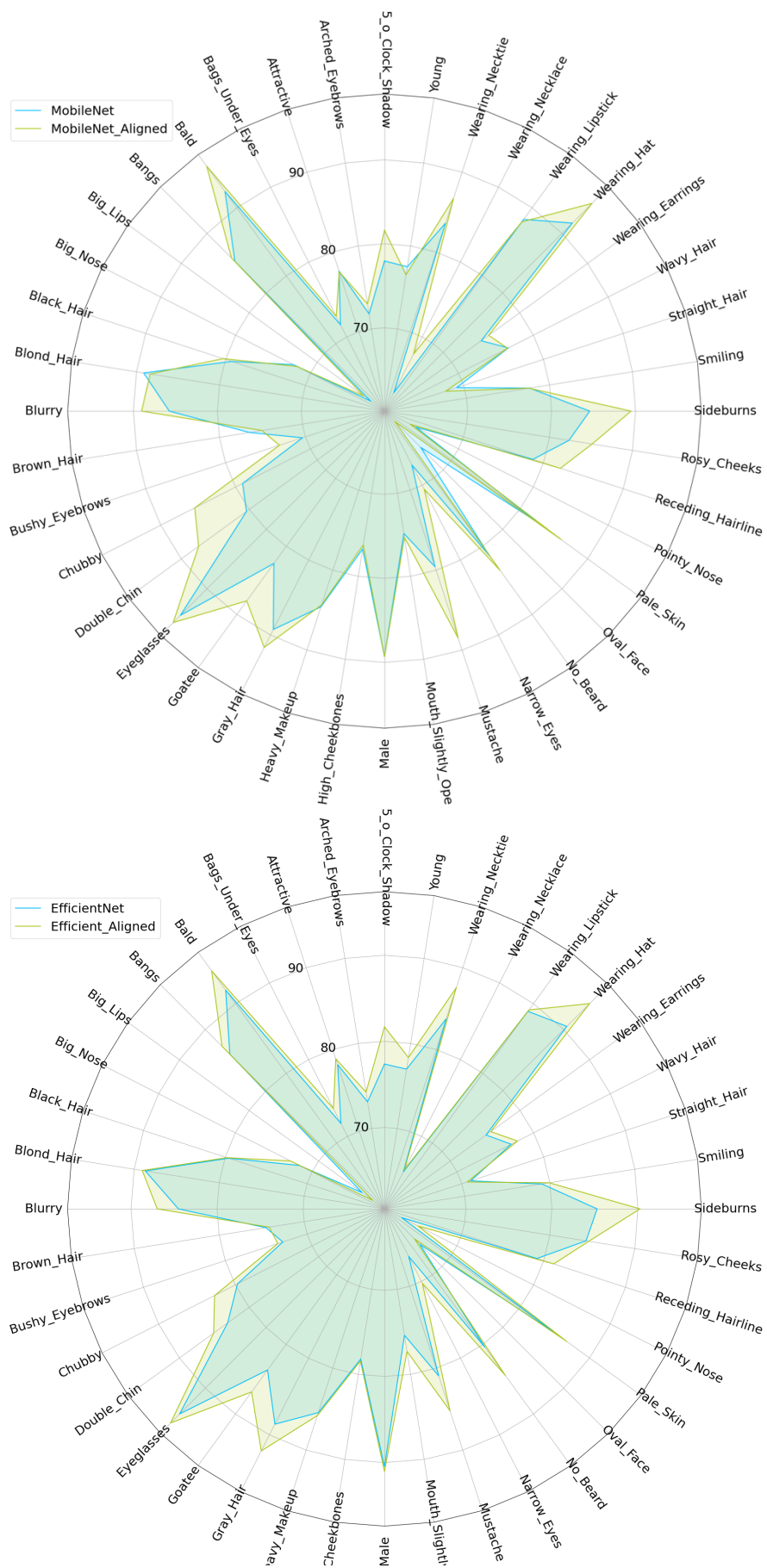


Figure 14: Label-wise accuracies of MobileNet and EfficientNet.

Table 3: Evaluation metrics of models with and without face alignment

	Accuracy		Loss		F1 Score	
	Non-aligned	Aligned	Non-aligned	Aligned	Non-Aligned	Aligned
ShuffleNetV2	81%	82%	0.53	0.59	74%	75%
MobileNetV2	78%	79%	0.62	0.70	69%	70%
EfficientNet	78%	79%	0.62	0.62	69%	71%

4.6.2 Complete fine-tuning

In these experiments, the backbone CNN models are fine-tuned as well as the classifier heads. The models were trained with aligned facial images. These models will also be compared to a ResNet-50 model.

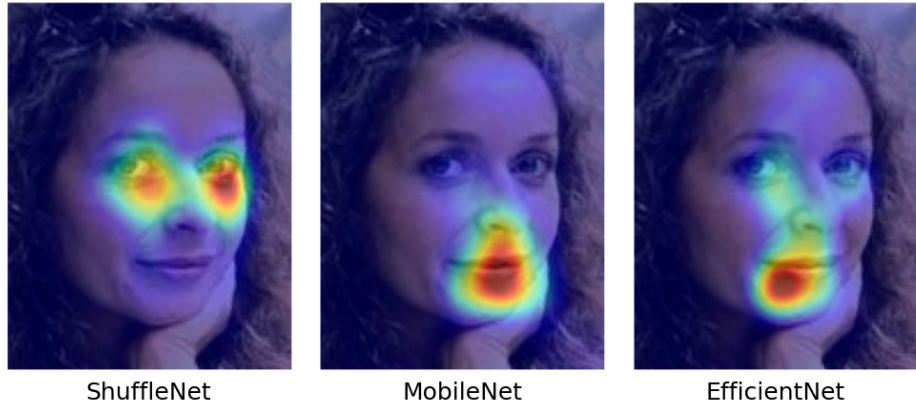


Figure 15: Activation maps of different models

Figure 15 shows that by training the whole model changes drastically the activation maps of the models compared to the activations that were produced by the models for which only the classifier heads were trained. On Figure 11, only the ShuffleNet showed activations on the face, but now every model has activation on it.

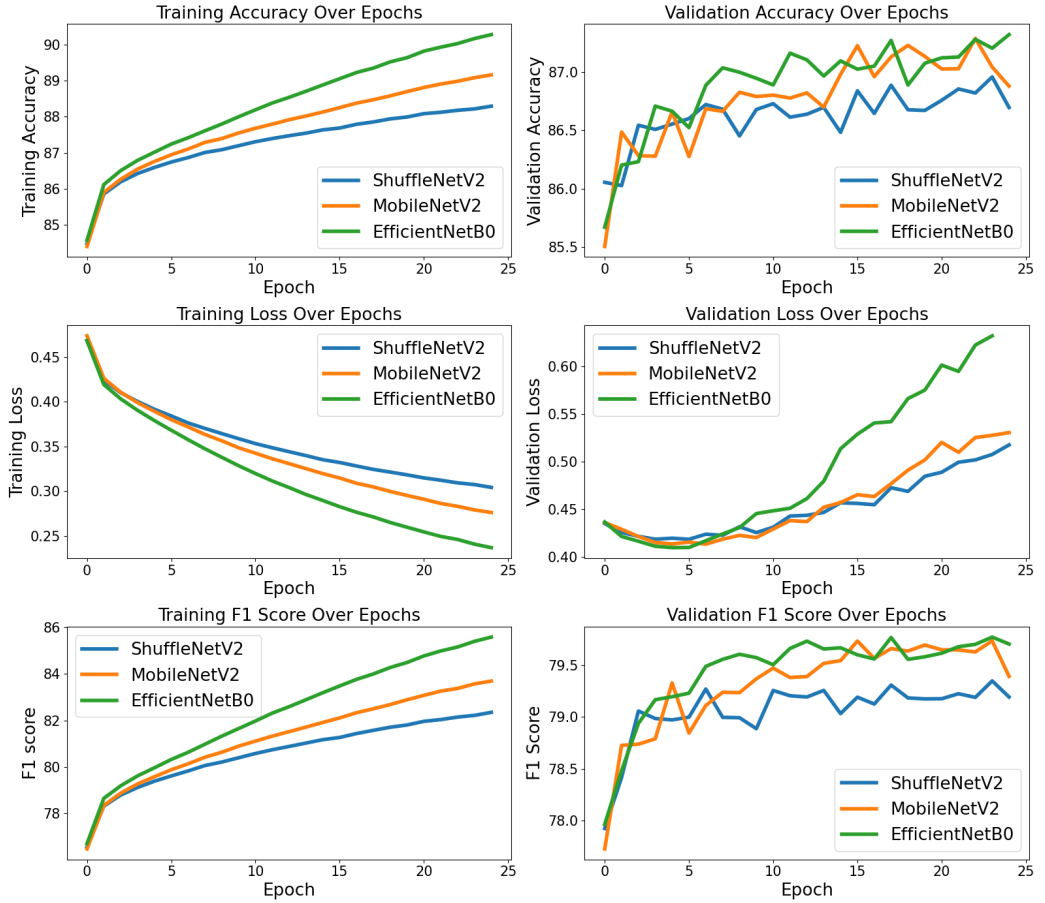


Figure 16: Training and validation curves

Figure 16 shows the three models' evaluation metrics. When training the backbone CNN models too, EfficientNet shows the strongest training performance in every metric, followed by MobileNet. But despite the great training performance the validation curves indicate overfitting. This is most obvious in the validation loss. Every model achieves their lowest loss at around the fifth epoch; from then on, all of them start to increase. The other signs of overfitting are the sizable gaps between the training and validation metrics, the latter also converges fast to a specific range of values, then starts to fluctuate. Of the three, the MobileNet demonstrates the best, most consistent result. Its validation metrics are there with the EfficientNet, however, its loss is around the same as the ShuffleNet's. One possible reason for the EfficientNet's immense overfitting, despite using regularization techniques, such as dropout, is its number of parameters. Because of its parameters, it is capable of fitting the training data, yet fails to generalize, so it requires additional regularization techniques or more data. The other two also show overfitting, but their capacity is smaller, so they generalize better. Table 4 shows the results of all three models. It also shows the results for a ResNet-50 for comparison.

Table 4: Comparison of evaluation metrics of the models

<i>Metrics</i>	<i>Trained Module</i>	<i>ShuffleNet</i>	<i>MobileNet</i>	<i>EfficientNet</i>	<i>Resnet-50</i>
<i>Accuracy</i>	CNN + Classifier	86.15%	86.42%	86.73%	86.77%
	Classifier	81.90%	78.03%	79.14%	79.84%
<i>Hamming Loss</i>	CNN + Classifier	0.12	0.11	0.11	0.10
	Classifier	0.16	0.19	0.19	0.17
<i>Precision</i>	CNN + Classifier	76.11%	76.27%	77.11%	78.09%
	Classifier	70.74%	66.49%	68.07%	68.64%
<i>Recall</i>	CNN + Classifier	85.37%	85.52%	84.09%	79.53%
	Classifier	81.70%	77.06%	78.55%	77.99%
<i>F1 Score</i>	CNN + Classifier	79.05%	79.33%	79.38%	78.06%
	Classifier	73.89%	69.26%	70.51%	71.20%

4.6.3 Correlation between metrics and positive samples

As shown in Figure 17, there is a correlation between the number of positive labels and the label-wise accuracy when only the classifier head was trained.

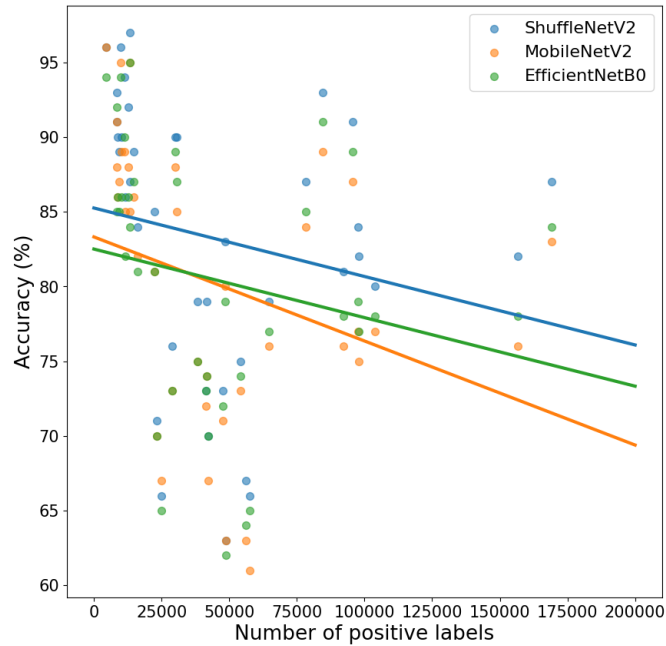


Figure 17: Correlation between the number of positive labels and label-wise accuracy.

Labels with fewer positive samples achieved greater accuracy, while labels with many positive samples achieved lower accuracy during evaluation, even when the loss function is weighted to reduce this effect. One possible reason is the imbalanced nature of the dataset. For labels that are less common, the model learned to predict the absence of these labels and therefore, during evaluation, it is biased towards predicting that the specific attribute is not present and reaches great results despite learning any meaningful information. For attributes with many positive samples, the model behaves the same, it learned to predict the presence of the labels. Meanwhile, for labels that have around the same number of positive and negative samples, they achieve similar results to the above-mentioned two cases, but the model also learned to generalize. Labels that are in between are mostly the hair colors (blonde hair, black hair, etc.) and lip and nose characteristics. These attributes are more unique to the given person than attributes like “pale skin” or “no beard”. Therefore, for the models is harder to generalize and also harder to overfit to that given attribute. This behavior is not unique to models where only the classifier heads were trained, but also to the models that were completely fine-tuned. This can be seen in Figure 18. Similarly to Figure 17, the label-wise accuracies decrease as the number of positive samples increases.

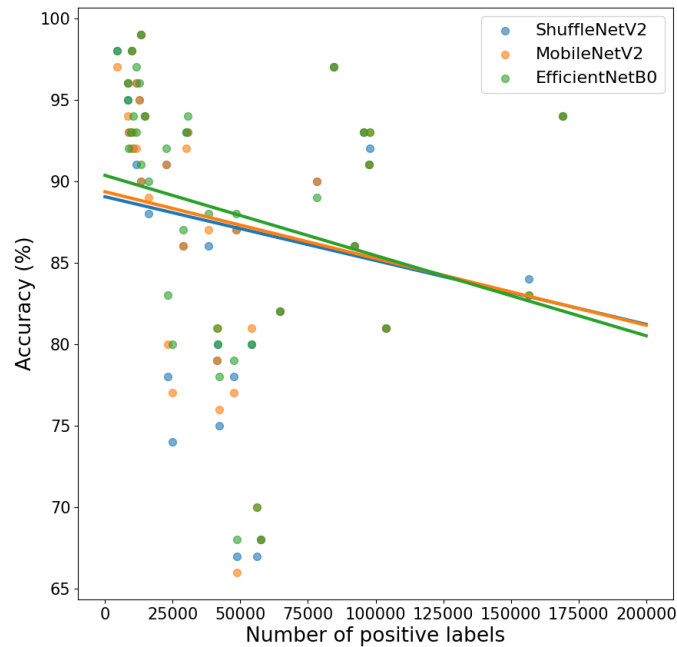


Figure 18: Correlation between label-wise accuracy and number of positive samples for completely fine-tuned models

In contrast to the correlation between label-wise accuracy and the number of positive samples, if we see the F1 score instead of the accuracy, the correlation changes

drastically. This is because the F1 score accounts for both the false positives and false negatives by being the harmonic mean of the precision and recall scores, and this makes it more suitable for evaluating an imbalanced classification problem. This can be seen in Figure 19 and Figure 20. As the number of positive samples is increasing, the F1 score is increasing with it.

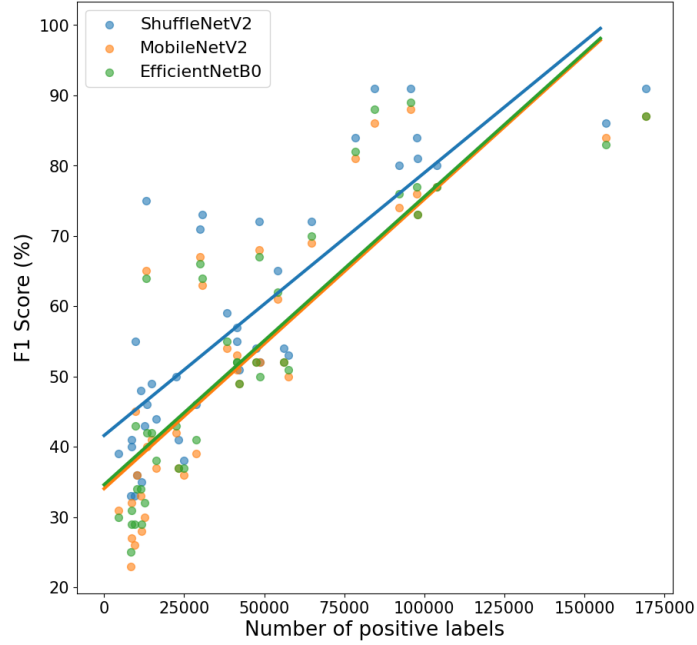


Figure 19: Correlation between label-wise F1 score and number of positive samples

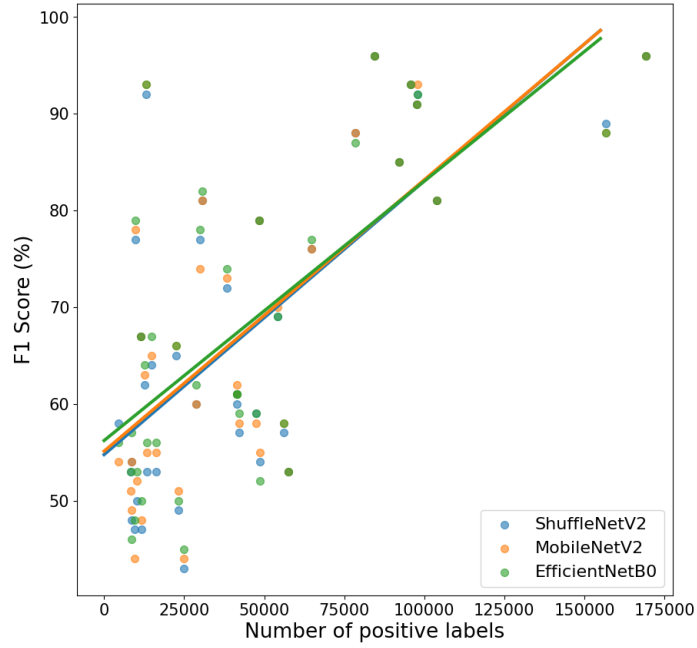


Figure 20: Correlation between label-wise F1 score and number of positive labels for models that are completely fine-tuned

4.6.4 Inference

In this section, I summarize the models' inference times and show examples of the image and video processing results. Table 5 contains the inference times of the models on single images on which the faces were already detected.

Table 5: Inference time of different models

<i>Models</i>	<i>Inference time</i>			
	<i>Latency (ms)</i>		<i>Throughput (samples/second)</i>	
<i>Hardware</i>	<i>CPU</i>	<i>GPU</i>	<i>CPU</i>	<i>GPU</i>
<i>ShuffleNet</i>	16.29	8.36	61.40	119.64
<i>MobileNetV2</i>	46.16	8.32	21.66	120.26
<i>EfficientNetB0</i>	209.17	11.79	4.78	84.85
<i>Resnet50</i>	39.82	9.08	25.11	110.16

ShuffleNet has the best inference time both on CPU and GPU. ResNet-50 is also a well-performing model despite the number of parameters. On CPU, the EfficientNet

performance is slow compared to the rest. It seems more suited to GPU deployment, where its inference time is only around 12 milliseconds. However, the main difference between the models is their memory size, which can be a bottleneck in IoT or other resource-constrained devices. Table 6 shows the size of the models in memory.

Table 6: Models' size in memory (MB)

<i>Models</i>	<i>Size in memory (MB)</i>
<i>ShuffleNet</i>	3.423
<i>MobileNet</i>	11.202
<i>EfficientNet</i>	18.036
<i>ResNet-50</i>	93.967

When processing an image or a frame of a video in practice, face detection is necessary because there could be more than one face. As stated in Paragraph 4.3, by default, the pipeline uses a YOLO based face detector. Its detection time ranges between 35 and 38 milliseconds, depending on the number of faces in the image. For demonstrational purposes, I used the MobileNet model on the image shown in Figure 21. Figure 22 and Figure 23 shows the detected faces, the predicted attributes and their corresponding confidence scores.



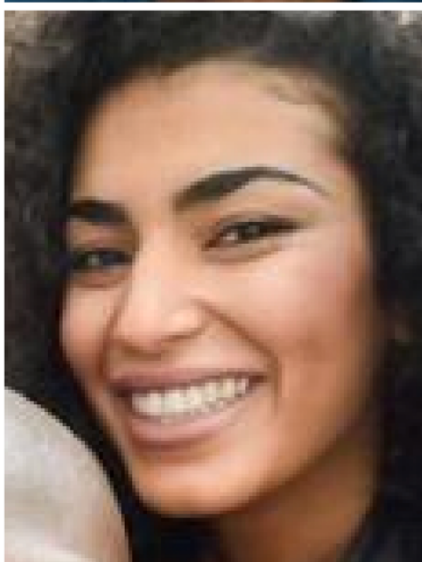
Figure 21: Test image [39]



Attributes	Confidence
Arched_Eyebrows	80.83%
Attractive	95.74%
Bags_Under_Eyes	62.94%
Big_Lips	55.25%
Big_Nose	73.54%
Brown_Hair	79.54%
Bushy_Eyebrows	96.08%
Heavy_Makeup	97.98%
High_Cheekbones	99.98%
Mouth_Slightly_Open	100.0%
No_Beard	99.96%
Oval_Face	78.18%
Smiling	100.0%
Straight_Hair	74.48%
Wearing_Earrings	83.75%
Wearing_Lipstick	99.24%
Young	99.63%
Female	100.0%

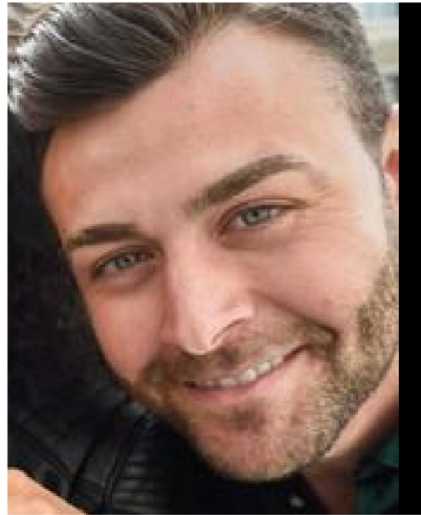


Attributes	Confidence
Arched_Eyebrows	59.77%
Bags_Under_Eyes	91.33%
Bald	74.55%
Big_Lips	95.79%
Big_Nose	99.86%
Bushy_Eyebrows	99.16%
High_Cheekbones	99.61%
Male	100.0%
Mouth_Slightly_Open	100.0%
Narrow_Eyes	95.16%
No_Beard	89.3%
Smiling	99.99%
Young	83.18%

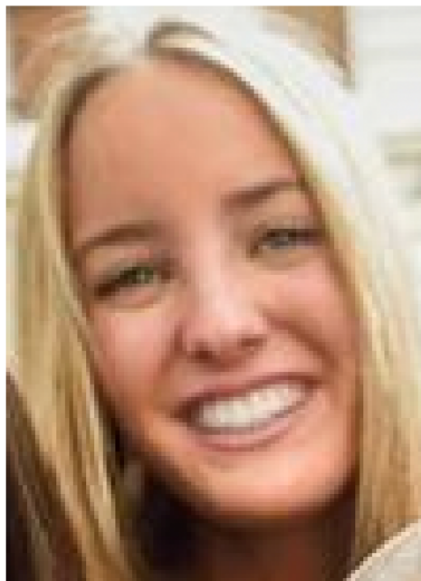


Attributes	Confidence
Arched_Eyebrows	63.69%
Attractive	76.07%
Bags_Under_Eyes	76.93%
Big_Lips	61.3%
Big_Nose	73.72%
Black_Hair	98.63%
Bushy_Eyebrows	95.16%
Heavy_Makeup	67.74%
High_Cheekbones	99.82%
Mouth_Slightly_Open	99.9%
Narrow_Eyes	96.0%
No_Beard	99.96%
Smiling	100.0%
Wavy_Hair	73.13%
Wearing_Lipstick	81.36%
Young	99.05%
Female	100.0%

Figure 22: Predicted attributes and their corresponding confidence scores.



Attributes	Confidence
5_o_Clock_Shadow	93.31%
Arched_Eyebrows	59.32%
Bags_Under_Eyes	94.29%
Big_Nose	92.64%
Bushy_Eyebrows	99.18%
Goatee	88.89%
Male	99.98%
Mustache	97.72%
Narrow_Eyes	56.21%
Pointy_Nose	67.43%
Receding_Hairline	85.68%
Sideburns	99.81%
Smiling	90.23%



Attributes	Confidence
Attractive	58.05%
Blond_Hair	97.73%
High_Cheekbones	91.1%
Mouth_Slightly_Open	99.98%
Narrow_Eyes	65.12%
No_Beard	99.95%
Oval_Face	62.3%
Smiling	99.95%
Straight_Hair	97.14%
Wearing_Lipstick	64.22%
Young	98.1%
Female	100.0%

Figure 23: Predicted attributes and their corresponding confidence scores.

5 Summary

In this thesis, with these experiments, I wanted to see the performance of efficient models in different training scenarios.

Using a pre-trained CNN as a feature extractor on a facial attribute recognition task and only fine-tuning a classifier head leads to non-comparative results. It has also been clear that the face alignment step is necessary in such facial downstream-tasks like facial attribute recognition because it can increase the model's performance by around one percent in most metrics. The experiments also showed that if the feature extractor is trained as well, it leads to great results, however, even these models with a relatively few number of parameters are prone to overfitting.

Convolutional neural networks can achieve great results on facial attribute recognition, but most state-of-the-art CNN FAR models include domain-specific knowledge. Currently, many advancements have been made in multi-attribute learning and in FAR and in general by using language models and autoencoders. These are universal multi-attribute frameworks that can be used for facial attribute recognition. These models are more often than not performing better than traditional CNN networks, but often require more resources to operate, but further research and advancements will probably overcome this issue.

References

- [1] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” Apr. 10, 2015, *arXiv*: arXiv:1409.1556. Accessed: Nov. 05, 2024. [Online]. Available: <http://arxiv.org/abs/1409.1556>
- [2] W. Luo, Y. Li, R. Urtasun, and R. Zemel, “Understanding the Effective Receptive Field in Deep Convolutional Neural Networks,” Jan. 25, 2017, *arXiv*: arXiv:1701.04128. Accessed: Nov. 05, 2024. [Online]. Available: <http://arxiv.org/abs/1701.04128>
- [3] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, “Squeeze-and-Excitation Networks,” May 16, 2019, *arXiv*: arXiv:1709.01507. Accessed: Nov. 19, 2024. [Online]. Available: <http://arxiv.org/abs/1709.01507>
- [4] A. Sharma and H. Foroosh, “Slim-CNN: A Light-Weight CNN for Face Attribute Prediction,” Jul. 03, 2019, *arXiv*: arXiv:1907.02157. Accessed: Nov. 20, 2024. [Online]. Available: <http://arxiv.org/abs/1907.02157>
- [5] X. Zhang, X. Zhou, M. Lin, and J. Sun, “ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices,” Dec. 07, 2017, *arXiv*: arXiv:1707.01083. Accessed: Oct. 24, 2024. [Online]. Available: <http://arxiv.org/abs/1707.01083>
- [6] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, “ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design,” Jul. 30, 2018, *arXiv*: arXiv:1807.11164. Accessed: Oct. 24, 2024. [Online]. Available: <http://arxiv.org/abs/1807.11164>
- [7] N. Liu, F. Zhang, L. Chang, and F. Duan, “Scattering-based hybrid network for facial attribute classification,” *Front. Comput. Sci.*, vol. 18, no. 3, p. 183313, Jun. 2024, doi: [10.1007/s11704-023-2570-6](https://doi.org/10.1007/s11704-023-2570-6).
- [8] M. Lin, Q. Chen, and S. Yan, “Network In Network,” Mar. 04, 2014, *arXiv*: arXiv:1312.4400. Accessed: Nov. 05, 2024. [Online]. Available: <http://arxiv.org/abs/1312.4400>
- [9] R. Al-Otaibi, P. Flach, and M. Kull, “Multi-label Classification: A Comparative Study on Threshold Selection Methods,” Sep. 2014.

- [10] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” Mar. 21, 2019, *arXiv*: arXiv:1801.04381. Accessed: Nov. 12, 2024. [Online]. Available: <http://arxiv.org/abs/1801.04381>
- [11] A. G. Howard *et al.*, “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” Apr. 17, 2017, *arXiv*: arXiv:1704.04861. Accessed: Nov. 06, 2024. [Online]. Available: <http://arxiv.org/abs/1704.04861>
- [12] Y. Shu, Y. Yan, S. Chen, J.-H. Xue, C. Shen, and H. Wang, “Learning spatial-semantic relationship for facial attribute recognition with limited labeled data,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 11916–11925.
- [13] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, “Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments,” University of Massachusetts, Amherst, 07–49, Oct. 2007.
- [14] D.-N. Le, V. S. Parvathy, D. Gupta, A. Khanna, J. J. P. C. Rodrigues, and K. Shankar, “IoT enabled depthwise separable convolution neural network with deep support vector machine for COVID-19 diagnosis and classification,” *Int. J. Mach. Learn. & Cyber.*, vol. 12, no. 11, pp. 3235–3248, Nov. 2021, doi: [10.1007/s13042-020-01248-7](https://doi.org/10.1007/s13042-020-01248-7).
- [15] J. Jin, A. Dundar, and E. Culurciello, “Flattened Convolutional Neural Networks for Feedforward Acceleration,” Nov. 20, 2015, *arXiv*: arXiv:1412.5474. Accessed: Nov. 12, 2024. [Online]. Available: <http://arxiv.org/abs/1412.5474>
- [16] K. Kärkkäinen and J. Joo, “FairFace: Face Attribute Dataset for Balanced Race, Gender, and Age,” Aug. 14, 2019, *arXiv*: arXiv:1908.04913. doi: [10.48550/arXiv.1908.04913](https://doi.org/10.48550/arXiv.1908.04913).
- [17] M. Tan and Q. V. Le, “EfficientNetV2: Smaller Models and Faster Training,” Jun. 23, 2021, *arXiv*: arXiv:2104.00298. Accessed: Oct. 27, 2024. [Online]. Available: <http://arxiv.org/abs/2104.00298>
- [18] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” Sep. 11, 2020, *arXiv*: arXiv:1905.11946. Accessed: Oct. 16, 2024. [Online]. Available: <http://arxiv.org/abs/1905.11946>

- [19] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” Mar. 09, 2015, *arXiv*: arXiv:1503.02531. Accessed: Nov. 12, 2024. [Online]. Available: <http://arxiv.org/abs/1503.02531>
- [20] L. Mao, Y. Yan, J.-H. Xue, and H. Wang, “Deep Multi-task Multi-label CNN for Effective Facial Attribute Classification,” Feb. 10, 2020, *arXiv*: arXiv:2002.03683. Accessed: Nov. 19, 2024. [Online]. Available: <http://arxiv.org/abs/2002.03683>
- [21] Z. Liu, P. Luo, X. Wang, and X. Tang, “Deep Learning Face Attributes in the Wild,” in *Proceedings of International Conference on Computer Vision (ICCV)*, Dec. 2015.
- [22] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [23] S. Coulibaly, B. Kamsu-Foguem, D. Kamissoko, and D. Traore, “Deep Convolution Neural Network sharing for the multi-label images classification,” *Machine Learning with Applications*, vol. 10, p. 100422, Dec. 2022, doi: [10.1016/j.mlwa.2022.100422](https://doi.org/10.1016/j.mlwa.2022.100422).
- [24] A. Araujo, W. Norris, and J. Sim, “Computing Receptive Fields of Convolutional Neural Networks,” *Distill*, vol. 4, no. 11, p. 10.23915/distill.00021, Nov. 2019, doi: [10.23915/distill.00021](https://doi.org/10.23915/distill.00021).
- [25] E. M. Hand and R. Chellappa, “Attributes for Improved Attributes: A Multi-Task Network for Attribute Classification,” Apr. 25, 2016, *arXiv*: arXiv:1604.07360. doi: [10.48550/arXiv.1604.07360](https://doi.org/10.48550/arXiv.1604.07360).
- [26] Zhang Zhifei, Song, Yang and H. Qi, “Age Progression/Regression by Conditional Adversarial Autoencoder,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2017.
- [27] A. N. Tarekegn, M. Giacobini, and K. Michalak, “A review of methods for imbalanced multi-label classification,” *Pattern Recognition*, vol. 118, p. 107965, Oct. 2021, doi: [10.1016/j.patcog.2021.107965](https://doi.org/10.1016/j.patcog.2021.107965).
- [28] J. Deng, J. Guo, Y. Zhou, J. Yu, I. Kotsia, and S. Zafeiriou, “RetinaFace: Single-stage Dense Face Localisation in the Wild,” May 04, 2019, *arXiv*: arXiv:1905.00641. doi: 10.48550/arXiv.1905.00641.

- [29] W. Li, Z. Cao, J. Feng, J. Zhou, and J. Lu, "Label2Label: A Language Modeling Framework for Multi-Attribute Learning," Jul. 18, 2022, arXiv: arXiv:2207.08677. doi: 10.48550/arXiv.2207.08677.
- [30] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks," *IEEE Signal Process. Lett.*, vol. 23, no. 10, pp. 1499–1503, Oct. 2016, doi: 10.1109/LSP.2016.2603342.
- [31] Z. Cai et al., "MARLIN: Masked Autoencoder for facial video Representation LearnINg," Mar. 22, 2023, arXiv: arXiv:2211.06627. doi: 10.48550/arXiv.2211.06627.
- [32] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," May 09, 2016, arXiv: arXiv:1506.02640. doi: 10.48550/arXiv.1506.02640.
- [33] C. Lei *et al.*, "AI-assisted facial analysis in healthcare: From disease detection to comprehensive management," *Patterns*, vol. 6, no. 2, p. 101175, Feb. 2025, doi: [10.1016/j.patter.2025.101175](https://doi.org/10.1016/j.patter.2025.101175).
- [34] X. Zhang, T. Wang, J. Ji, Y. Zhang, and R. Lan, "Privacy-preserving face attribute classification via differential privacy," *Neurocomputing*, vol. 626, p. 129556, Apr. 2025, doi: [10.1016/j.neucom.2025.129556](https://doi.org/10.1016/j.neucom.2025.129556).
- [35] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. J. Burges, L. Bottou, and K. Q. Weinberger, Eds., Curran Associates, Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf
- [36] GeeksForGeeks: Multi-Layer Perceptron Learning in Tensorflow <https://www.geeksforgeeks.org/multi-layer-perceptron-learning-in-tensorflow/> (Accessed: May. 21, 2024)
- [37] 8th Lights: What Is a Convolution? How To Teach Machines To See Images <https://8thlight.com/insights/what-is-a-convolution-how-to-teach-machines-to-see-images> (Accessed: May. 21, 2024)

- [38] Medium: Receptive Field
<https://medium.com/@saba99/receptive-field-1726fe6ea94f>
(Accessed: May. 21, 2024)
- [39] Dot To Dot: Influencer marketing has exploded over the recent years.
<https://dottodot.co.za/influencer-marketing/>
(Accessed: May. 21, 2025)
- [40] Medium: A Look at MobileNetV2: Inverted Residuals and Linear Bottlenecks
https://medium.com/@luis_gonzales/a-look-at-mobilenetv2-inverted-residuals-and-linear-bottlenecks-d49f85c12423
(Accessed: May. 21, 2025)
- [41] Xailient: 4 Popular Model Compression Techniques Explained
https://xailient.com/blog/4-popular-model-compression-techniques-explained/#3_The_knowledge_distillation_technique
(Accessed: May. 21, 2025)